Theme Park Operations & Visitor Analytics

Comprehensive Azure Databricks hands-on assessment | 50 exercises + capstone | No solutions or marks

Objective: Use familiar Azure Databricks techniques to clean, transform, govern, and analyze visitor entries and attraction operations. Build a working Delta Lake workflow and an incremental Bronze-Silver-Gold pipeline.

Coverage: SQL, PySpark DataFrames, Delta tables, CRUD, MERGE / UPSERT, SCD Type 1 and Type 2, transaction history, Time Travel, VACUUM, OPTIMIZE, Parquet conversion, views, Structured Streaming, Lakeflow pipelines, and charts.

Assessment files

File	Purpose
park_entries.csv	121 visitor-entry rows; intentional quality issues
attractions.json	10 attractions; nested location and feature arrays
attraction_maintenance.csv	28 maintenance records
park_entry_changes.csv	10 incoming rows for MERGE / UPSERT
park_entries_incremental.csv	12 new rows for a later streaming update
databricks_dataset_setup.py	Databricks setup notebook; no exercise solutions

Getting started

- Upload the five data files into one folder in a Unity Catalog Volume.
- Import the supplied Databricks setup notebook and edit CATALOG, SCHEMA, and BASE_PATH.
- Run setup once. It creates four Delta tables: tp_entries_raw, tp_attractions, tp_maintenance, and tp_entry_changes.
- Keep tp_entries_raw append-only. Use a separate working table for UPDATE, DELETE, and MERGE.
- Use pipeline source files (SQL or Python) for Lakeflow exercises, not ordinary notebook SQL cells.
- Only append the incremental CSV after the first successful pipeline update.

Required: Azure Databricks workspace, Unity Catalog catalog/schema with table-creation permissions, a writable Volume, and supported compute for streaming and Lakeflow pipelines.

A. Data ingestion and exploration

01. Load the supplied CSV and JSON datasets using the setup notebook. Display each table and verify its row count.
02. Inspect the schema of tp_entries_raw. Identify string, integer, and decimal columns.
03. Select entry_id, attraction_id, visit_date, ticket_type, and ticket_price. Display the first 20 rows.
04. Using SQL, find Completed entries where ticket_price is greater than 500.
05. Use DISTINCT and IN to examine ticket types and booking channels.
06. Convert visit_date to a date type and count entries by visit date.

B. Cleaning and transforming with PySpark

07. Find records with missing ticket_price or booking_channel values.
08. Fill missing booking_channel values with Unknown in a new DataFrame; do not change the raw source.
09. Identify records with negative wait_minutes, zero ride_minutes, or nonpositive ticket_price.
10. Find duplicate entry_id values and create a deduplicated DataFrame.
11. Identify entry records whose attraction_id is missing from the attraction master.
12. Create wait_category: Short (0-20), Medium (21-45), Long (above 45). Exclude invalid waits.
13. Join entries with attractions and extract location.zone and features from the nested JSON data.

C. SQL joins and aggregations

14. Find Completed entries for Standard or Express tickets using WHERE, IN, AND, and BETWEEN.
15. Calculate total valid ticket revenue and completed-entry counts by attraction.
16. Calculate average waiting time by attraction category, excluding invalid waits.
17. Join maintenance records with attractions and calculate completed-maintenance downtime by attraction.
18. Calculate total completed-entry revenue by park zone using the nested location.zone attribute.

D. Delta table creation and CRUD

19. Create a clean managed Delta table named tp_entries_work using CTAS or a PySpark DataFrame write.
20. Create a second Delta table from a PySpark DataFrame using saveAsTable().
21. Insert two new valid visitor entries into tp_entries_work using SQL.
22. Update the status and ticket price of an existing entry in tp_entries_work.
23. Delete a Cancelled entry from tp_entries_work and verify the result.
24. Use DESCRIBE EXTENDED to identify the working table type and location; explain managed versus external storage.

E. MERGE / UPSERT and SCD

25. Inspect tp_entry_changes and separate existing entry IDs from new IDs.

26. Use SQL MERGE to update matching entries and insert new entries into tp_entries_work (SCD Type 1).
27. Verify which entry IDs were updated and which were inserted.
28. Create a separate Delta table and perform the same MERGE using the PySpark DeltaTable API.
29. Create an attraction-capacity history table with valid_from, valid_to, and is_current columns.
30. Implement an SCD Type 2 change for one attraction capacity; query both its historical and current rows.

F. Delta internals, Time Travel, and optimization

31. Use DESCRIBE DETAIL and DESCRIBE HISTORY on tp_entries_work. Record the storage format and operations.
32. Find the table version immediately before your MERGE and query it using VERSION AS OF.
33. Compare one updated entry before and after the MERGE.
34. Run OPTIMIZE, then compare the file count before and after. Explain why small tables may not change.
35. Run ZORDER BY (attraction_id) if your table is not liquid clustered, and inspect the history.
36. Explain VACUUM retention and perform a safe dry run if supported; do not delete historical files.

G. Views and Parquet

37. Create a permanent view of valid Completed entries joined with attraction names.
38. Create a temporary view of entries with wait_minutes greater than 45.
39. Create and query a global temporary view if supported by your compute.
40. Write a DataFrame to Parquet in an approved Volume or external location; read it back and inspect its schema.
41. Convert that Parquet data to a new Delta table by reading it and writing Delta; explain what the Delta transaction log adds.

H. Structured Streaming

42. Create a streaming DataFrame from Spark rate with five rows per second.
43. Create visitor_event_id and simulated_wait_minutes columns from the streaming value.
44. Filter the stream to retain simulated waits above 30 minutes.
45. Aggregate streaming events into two wait categories, display results with a supported sink, and stop the query.

I. Lakeflow / DLT and incremental loading

46. Create a SQL Lakeflow pipeline with a Bronze streaming table reading append-only tp_entries_raw.
47. Create a Silver streaming table that retains valid Completed entries and calculates ticket revenue.
48. Create a Gold materialized view showing entry counts and revenue by attraction; run and inspect the pipeline graph.

49. Create a separate Python Lakeflow pipeline for the same Bronze-Silver-Gold flow using distinct output names.
50. Append park_entries_incremental.csv to tp_entries_raw only after the first pipeline run; rerun the pipeline and verify the Gold totals.

Final capstone - Park Operations Dashboard

Build a complete, repeatable visitor-analytics workflow using the supplied datasets. Use the same techniques practiced in the assessment.

• Bronze: ingest the append-only visitor-entry Delta source.

• Silver: validate records, handle missing channels, and calculate revenue.

• Gold: aggregate entries, revenue, and average waiting time by attraction and zone.

• Create a bar chart for revenue by attraction, a pie chart for ticket-type share, a line chart for daily revenue, and a scatter plot comparing average wait and revenue.

• Demonstrate a successful MERGE, one historical table version, and a second incremental pipeline update.

• Explain how you would separate development and production schemas and pipeline configurations.

Submission checklist

☐ SQL notebook and PySpark notebook

☐ SQL and Python Lakeflow pipeline source files

☐ Screenshots of pipeline graph and Gold outputs

☐ Four charts and a short findings summary

☐ A short note describing data-quality issues, Time Travel, and incremental loading

Assessment rules and safety notes

• No solutions are supplied. Write and run your own code for every task.

• Use separate working tables for CRUD and MERGE; keep the streaming source append-only until the incremental exercise.

• Do not recreate the source Delta table after the pipeline begins, because doing so may invalidate streaming progress.

• For external tables or in-place Parquet conversion, use only approved external locations, never internal __unitystorage paths.

• Global temporary views, ZORDER, and streaming sinks depend on compatible Databricks compute and table settings.

• Do not run destructive VACUUM before completing Time Travel tasks.